

Ex. No: 12

Case Study:

Banking is one of the most familiar real-life database applications. A bank must keep a record of its customers and their accounts, make sure every account balance stays consistent, and log each deposit and withdrawal the moment it happens. This case study models a simplified retail banking system, in which a customer holds one or more accounts and every movement of money is recorded as a transaction, and shows how a desktop front end can carry out these operations directly against the database.

Entities and Relationships:

The design is built from three entities. Customer (custid, cname, phone) represents the account holder. Account (accno, custid, balance) belongs to exactly one customer through a foreign key on custid, and its balance is constrained never to fall below zero. Transactions (txnid, accno, txntype, amount, balance_after, txntime) records every operation performed on an account, whether OPEN, DEPOSIT or WITHDRAW, and is linked to Account by a foreign key. The relationships are therefore one Customer to many Accounts, and one Account to many Transactions.

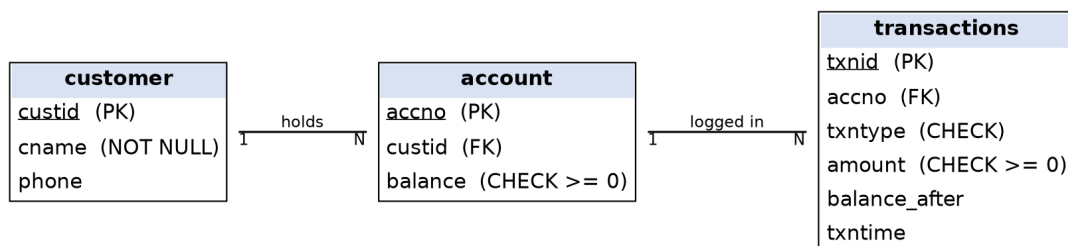


Fig. 12.1: ER diagram (customer 1:N account 1:N transactions).

The tables are in Third Normal Form: customer details are stored once, and each transaction refers to its account only through accno. transactions.balance_after is a deliberate snapshot of the balance, so a statement can be shown without replaying the ledger.

Features:

The application supports six operations: Insert (open a new account together with its customer), Update (edit a customer's name and phone), Delete (close an account and remove its transaction history), Deposit and Withdraw (each guarded by an insufficient-balance check), and History (view the transaction log of a single account or of all accounts). A balance can change only through Deposit or Withdraw, so the account balance and the transaction ledger always stay in step.

Database Connectivity:

The front end is built with Tkinter, Python's standard GUI toolkit, and connects to the SQLite database through Python's built-in sqlite3 module. Every button on the window issues SQL over this connection: for example, Deposit runs an UPDATE on account and then an INSERT into transactions within a single database transaction, so both writes succeed together or neither is applied. Referential integrity is switched on with PRAGMA foreign_keys = ON, and table-level rules such as CHECK (balance >= 0) are enforced by the database engine itself, not by the interface. The schema and the complete program follow.

Architecture: BankApp (Tkinter) is the presentation layer and only calls BankDB methods; BankDB is the application layer that validates input and controls transactions; the sqlite3 driver (the role JDBC or ODBC plays in other stacks) carries SQL to the SQLite DBMS. Here all layers run on one computer; a real bank would put an application server between the app and the database, reached over HTTPS.

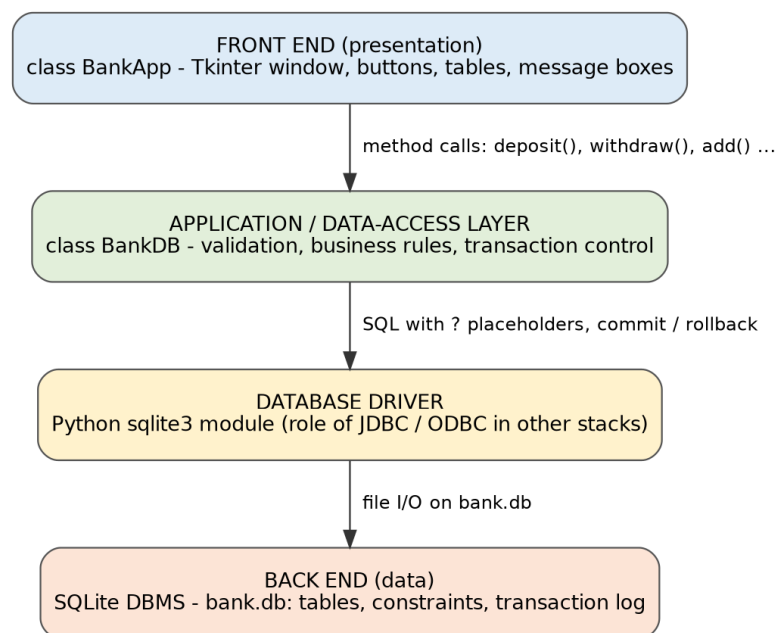


Fig. 12.2: Layers and connectivity path from the GUI to the database.

Schema (schema.sql):

```
CREATE TABLE IF NOT EXISTS customer (  
    custid INTEGER PRIMARY KEY AUTOINCREMENT,  
    cname TEXT NOT NULL,  
    phone TEXT  
);
```

```

CREATE TABLE IF NOT EXISTS account (
    accno INTEGER PRIMARY KEY,
    custid INTEGER NOT NULL REFERENCES customer(custid),
    balance REAL NOT NULL DEFAULT 0 CHECK (balance >= 0)
);

CREATE TABLE IF NOT EXISTS transactions (
    txnid INTEGER PRIMARY KEY AUTOINCREMENT,
    accno INTEGER NOT NULL REFERENCES account(accno),
    txntype TEXT NOT NULL CHECK (txntype IN ('OPEN', 'DEPOSIT',
'WITHDRAW')),
    amount REAL NOT NULL CHECK (amount >= 0),
    balance_after REAL NOT NULL,
    txntime TEXT NOT NULL DEFAULT (datetime('now', 'localtime'))
);

```

Program (bank_app.py):

```

import os
import sqlite3
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_FILE = os.path.join(BASE_DIR, "bank.db")
SCHEMA_FILE = os.path.join(BASE_DIR, "schema.sql")

class BankError(Exception):
    """Raised for any user-facing problem (bad input, no funds, ...)."""

class BankDB:
    def __init__(self, path=DB_FILE, schema_file=SCHEMA_FILE):
        self.conn = sqlite3.connect(path)
        self.conn.execute("PRAGMA foreign_keys = ON") # enforce foreign keys
        try:
            with open(schema_file, encoding="utf-8") as f:
                self.conn.executescript(f.read()) # run schema.sql
        except FileNotFoundError:
            raise SystemExit(
                f"schema.sql not found ({schema_file}).\n"
            )

```

```

cols = [r[1] for r in self.conn.execute("PRAGMA table_info(account)")]
if "custid" not in cols:
    raise SystemExit(
        "An old bank.db from the single-table version was found.\n"
        "Delete bank.db and run the program again."
    )

def all_accounts(self):
    return self.conn.execute(
        "SELECT a.accno, c.custid, c.cname, c.phone, a.balance "
        "FROM account a JOIN customer c ON a.custid = c.custid "
        "ORDER BY a.accno"
    ).fetchall()

def get(self, accno):
    row = self.conn.execute(
        "SELECT a.accno, c.custid, c.cname, c.phone, a.balance "
        "FROM account a JOIN customer c ON a.custid = c.custid "
        "WHERE a.accno = ?", (accno,)
    ).fetchone()
    if row is None:
        raise BankError(f"Account {accno} does not exist.")
    return row

def add(self, accno, cname, phone, balance):
    cname = cname.strip()
    balance = round(balance, 2)
    if not cname:
        raise BankError("Customer name cannot be empty.")
    if balance < 0:
        raise BankError("Opening balance cannot be negative.")
    exists = self.conn.execute(
        "SELECT 1 FROM account WHERE accno = ?", (accno,)
    ).fetchone()
    if exists:
        raise BankError(f"Account {accno} already exists.")
    try:
        with self.conn:
            cur = self.conn.execute(
                "INSERT INTO customer (cname, phone) VALUES (?, ?)",

```

```

        (cname, phone or None),
    )
    self.conn.execute(
        "INSERT INTO account (accno, custid, balance) VALUES (?, ?, ?)",
        (accno, cur.lastrowid, balance),
    )
    self.conn.execute(
        "INSERT INTO transactions (accno, txntype, amount, balance_after) "
        "VALUES (?, 'OPEN', ?, ?)",
        (accno, balance, balance),
    )
except sqlite3.IntegrityError as e:
    raise BankError(f"Could not save the record: {e}")

def update(self, accno, cname, phone):
    row = self.get(accno)
    cname = cname.strip()
    if not cname:
        raise BankError("Customer name cannot be empty.")
    with self.conn:
        self.conn.execute(
            "UPDATE customer SET cname = ?, phone = ? WHERE custid = ?",
            (cname, phone or None, row[1]),
        )

def delete(self, accno):
    row = self.get(accno)
    with self.conn:
        self.conn.execute("DELETE FROM transactions WHERE accno = ?", (accno,))
        self.conn.execute("DELETE FROM account WHERE accno = ?", (accno,))

        self.conn.execute(
            "DELETE FROM customer WHERE custid = ? "
            "AND NOT EXISTS (SELECT 1 FROM account WHERE custid = ?)",
            (row[1], row[1]),
        )

def deposit(self, accno, amount):
    return self._move(accno, amount, "DEPOSIT")

def withdraw(self, accno, amount):

```

```

        return self._move(accno, amount, "WITHDRAW")

def _move(self, accno, amount, kind):
    amount = round(amount, 2)
    if amount <= 0:
        raise BankError(f"{kind.title()} amount must be greater than zero.")
    self.get(accno)
    change = amount if kind == "DEPOSIT" else -amount
    with self.conn:
        cur = self.conn.execute(
            "UPDATE account SET balance = ROUND(balance + ?, 2) "
            "WHERE accno = ? AND balance + ? >= 0",
            (change, accno, change),
        )
        if cur.rowcount == 0:
            raise BankError("Insufficient balance.")
        new_balance = self.conn.execute(
            "SELECT balance FROM account WHERE accno = ?", (accno,)
        ).fetchone()[0]
        self.conn.execute(
            "INSERT INTO transactions (accno, txntype, amount, balance_after) "
            "VALUES (?, ?, ?, ?)",
            (accno, kind, amount, new_balance),
        )
    return new_balance

def history(self, accno=None):
    sql = ("SELECT txnid, accno, txntype, amount, balance_after, txntime "
           "FROM transactions ")
    if accno is None:
        return self.conn.execute(sql + "ORDER BY txnid DESC").fetchall()
    return self.conn.execute(
        sql + "WHERE accno = ? ORDER BY txnid DESC", (accno,)).fetchall()

def parse_accno(text):
    try:
        return int(text.strip())
    except ValueError:
        raise BankError("Account number must be a whole number.")

def parse_money(text, blank_ok=False):

```

```

text = text.strip()
if not text and blank_ok:
    return 0.0
try:
    return float(text)
except ValueError:
    raise BankError("Balance must be a number.")

def parse_phone(text):
    text = text.strip()
    if text and not (text.isdigit() and 7 <= len(text) <= 15):
        raise BankError("Phone number must be 7 to 15 digits.")
    return text

class BankApp(tk.Tk):
    def __init__(self, db):
        super().__init__()
        self.db = db
        self.title("Banking System")
        self.resizable(False, False)
        self.accno = tk.StringVar()
        self.cname = tk.StringVar()
        self.phone = tk.StringVar()
        self.balance = tk.StringVar()
        self._build()
        self.refresh()

    def _build(self):
        form = ttk.LabelFrame(self, text="Account details", padding=10)
        form.grid(row=0, column=0, padx=10, pady=(10, 4), sticky="ew")
        fields = [
            ("Account No", self.accno),
            ("Customer Name", self.cname),
            ("Phone", self.phone),
            ("Balance (Rs)", self.balance),
        ]
        for r, (label, var) in enumerate(fields):
            ttk.Label(form, text=label).grid(row=r, column=0, sticky="w", pady=3)
            ttk.Entry(form, textvariable=var, width=30).grid(row=r, column=1, padx=8)

```

```

buttons = ttk.Frame(self)
buttons.grid(row=1, column=0, padx=10, pady=4)
actions = [
    ("Insert", self.insert),
    ("Update", self.update),
    ("Delete", self.delete),
    ("Clear", self.clear),
    ("Deposit", self.deposit),
    ("Withdraw", self.withdraw),
    ("History", self.show_history),
    ("Exit", self.destroy),
]
for i, (text, cmd) in enumerate(actions):
    ttk.Button(buttons, text=text, command=cmd, width=10).grid(
        row=i // 4, column=i % 4, padx=3, pady=3
    )

cols = ("accno", "custid", "cname", "phone", "balance")
self.tree = ttk.Treeview(self, columns=cols, show="headings", height=8)
for col, title, width in [
    ("accno", "Account No", 90),
    ("custid", "Cust ID", 60),
    ("cname", "Customer Name", 160),
    ("phone", "Phone", 110),
    ("balance", "Balance (Rs)", 100),
]:
    self.tree.heading(col, text=title)
    self.tree.column(col, width=width, anchor="w")
self.tree.grid(row=2, column=0, padx=10, pady=(4, 10))
self.tree.bind("<<TreeviewSelect>>", self.on_select)
def refresh(self):
    self.tree.delete(*self.tree.get_children())
    for accno, custid, cname, phone, balance in self.db.all_accounts():
        self.tree.insert("", "end",
            values=(accno, custid, cname, phone or "", f'{balance:.2f}'))

def on_select(self, _event=None):
    sel = self.tree.selection()
    if sel:
        accno, _custid, cname, phone, balance = self.tree.item(sel[0], "values")
        self.accno.set(accno)

```

```

        self.cname.set(cname)
        self.phone.set(phone)
        self.balance.set(balance)

def error(self, err):
    messagebox.showerror("Banking System", str(err), parent=self)

def info(self, msg):
    messagebox.showinfo("Banking System", msg, parent=self)

def ask_amount(self, title, prompt):
    return simpledialog.askfloat(title, prompt, parent=self, minvalue=0.01)

def insert(self):
    try:
        accno = parse_accno(self.accno.get())
        balance = parse_money(self.balance.get(), blank_ok=True)
        phone = parse_phone(self.phone.get())
        self.db.add(accno, self.cname.get(), phone, balance)
    except BankError as e:
        return self.error(e)
    self.refresh()
    self.info("Record inserted successfully.")

def update(self):
    try:
        accno = parse_accno(self.accno.get())
        phone = parse_phone(self.phone.get())
        self.db.update(accno, self.cname.get(), phone)
    except BankError as e:
        return self.error(e)
    self.refresh()
    self.info("Record updated successfully. "
              "(Balance changes only through Deposit / Withdraw.)")

def delete(self):
    try:
        accno = parse_accno(self.accno.get())
        self.db.get(accno)
        if not messagebox.askyesno(
            "Banking System",

```

```

        f"Delete account {accno} and its transaction history?",
        parent=self):
    return
    self.db.delete(accno)
except BankError as e:
    return self.error(e)
self.clear()
self.refresh()
self.info("Record deleted.")

def clear(self):
    for var in (self.accno, self.cname, self.phone, self.balance):
        var.set("")
    self.tree.selection_remove(self.tree.selection())

def deposit(self):
    self._transact("Deposit", "Enter the amount to be deposited (Rs):",
        self.db.deposit)

def withdraw(self):
    self._transact("Withdraw", "Enter the amount to be withdrawn (Rs):",
        self.db.withdraw)

def _transact(self, title, prompt, operation):
    try:
        accno = parse_accno(self.accno.get())
        self.db.get(accno)
        amount = self.ask_amount(title, prompt)
        if amount is None:
            return
        new_balance = operation(accno, amount)
    except BankError as e:
        return self.error(e)
    self.refresh()
    self.balance.set(f"{new_balance:.2f}")
    self.info(f"Current balance is Rs {new_balance:.2f}")

def show_history(self):
    """Transaction history of the account in the box (all accounts if blank)."""
    try:
        text = self.accno.get().strip()

```

```

        accno = parse_accno(text) if text else None
        if accno is not None:
            self.db.get(accno)
            rows = self.db.history(accno)
        except BankError as e:
            return self.error(e)

win = tk.Toplevel(self)
win.title(f"Transaction history - account {accno}" if accno is not None
        else "Transaction history - all accounts")
win.resizable(False, False)
cols = ("txnid", "accno", "txntype", "amount", "balance_after", "txntime")
tree = ttk.Treeview(win, columns=cols, show="headings", height=12)
for col, title, width in [
    ("txnid", "Txn ID", 60),
    ("accno", "Account No", 90),
    ("txntype", "Type", 80),
    ("amount", "Amount (Rs)", 100),
    ("balance_after", "Balance After", 100),
    ("txntime", "Date & Time", 150),
]:
    tree.heading(col, text=title)
    tree.column(col, width=width, anchor="w")
for txnid, acc, kind, amount, after, when in rows:
    tree.insert("", "end",
        values=(txnid, acc, kind, f'{amount:.2f}', f'{after:.2f}', when))
tree.grid(row=0, column=0, padx=10, pady=10)
ttk.Button(win, text="Close", command=win.destroy).grid(row=1, column=0,
pady=(0, 10))

if __name__ == "__main__":
    BankApp(BankDB()).mainloop()

```

Output:

Banking System

Account details

Account No

235612786549

Customer Name

Akash

Phone

9751657835

Balance (Rs)

10000.00

Insert

Update

Delete

Clear

Deposit

Withdraw

History

Exit

Account No	Cust ID	Customer Name	Phone	Balance (Rs)
235612786549	1	Akash	9751657835	10000.00

Fig. 12.3: Main window after Insert, showing the new account with its opening balance of Rs 10,000.00.

Transaction history - account 235612786549

Txn ID	Account No	Type	Amount (Rs)	Balance After	Date & Time
3	235612786549	WITHDRAW	3000.00	9000.00	2026-09-30 12:09:40
2	235612786549	DEPOSIT	2000.00	12000.00	2026-09-30 12:09:33
1	235612786549	OPEN	10000.00	10000.00	2026-09-30 12:08:58

Close

Fig. 12.4: Transaction history of the same account: OPEN Rs 10,000.00, DEPOSIT Rs 2,000.00, WITHDRAW Rs 3,000.00, final balance Rs 9,000.00.

The screenshot shows a window titled "Banking System". Inside, there is a section labeled "Account details" with four input fields: "Account No" (235612786549), "Customer Name" (Akash Kumar), "Phone" (9751657835), and "Balance (Rs)" (9000.00). Below these fields are two rows of buttons: the first row contains "Insert", "Update", "Delete", and "Clear"; the second row contains "Deposit", "Withdraw", "History", and "Exit". At the bottom of the window is a table with the following data:

Account No	Cust ID	Customer Name	Phone	Balance (Rs)
235612786549	1	Akash Kumar	9751657835	9000.00

Fig. 12.5: Main window after Update: the customer name is changed to "Akash Kumar"; the balance stays Rs 9,000.00 because only Deposit and Withdraw change it.

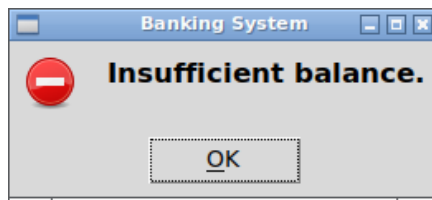


Fig. 12.6: Withdraw of Rs 20,000.00 is rejected with the "Insufficient balance." message; the balance is not changed.

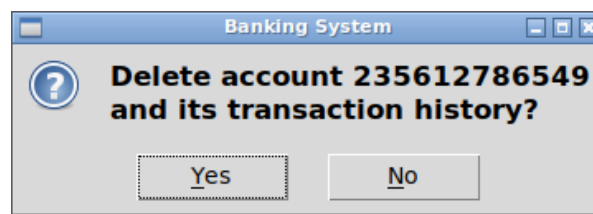


Fig. 12.7: Confirmation prompt shown when Delete is clicked.

The image shows a software window titled "Banking System". It contains a section for "Account details" with four input fields: "Account No", "Customer Name", "Phone", and "Balance (Rs)". Below these fields are two rows of buttons: "Insert", "Update", "Delete", "Clear" in the first row, and "Deposit", "Withdraw", "History", "Exit" in the second row. At the bottom is a table with five columns: "Account No", "Cust ID", "Customer Name", "Phone", and "Balance (Rs)". The table is currently empty.

Account No	Cust ID	Customer Name	Phone	Balance (Rs)
------------	---------	---------------	-------	--------------

Fig. 12.8: Main window after the account is deleted; the account list is empty.